

SHAYVERI CORE - Phase 0 Scaffold Summary

Compute Orchestration & Rendering Engine · Java / Spring Boot backend

Session recap · July 7, 2026

What This Document Is

A recap of everything done in the scaffolding session, before any real feature code (Module 1 onward) exists. The full build plan lives in README.md at the project root. Project location:

```
/Users/averi/Undergrad/CS/Java/Java projects/  
SHAYVERI CORE (Compute Orchestration & Rendering Engine)
```

1. What Was Built (Phase 0 Only)

Phase 0 is deliberately just the skeleton: the app boots, API-key security works, and one ping endpoint exists. No telemetry, jobs, nodes, or config logic has been written yet - that is Phases 1 through 4.

Files created

File	Purpose
build.gradle.kts / settings.gradle.kts	Gradle Kotlin DSL build. Spring Boot 3.5.0, Java 21 toolchain, starters: web, websocket, data-mongodb, data-redis, validation, actuator, security.
gradlew, gradlew.bat, gradle/wrapper/	Committed Gradle wrapper pinned to 8.14.2 so anyone can build without installing Gradle.
docker-compose.yml	Local dev infra: mongo:7 and redis:7-alpine, one command (docker compose up -d).
src/main/resources/application.yml	Mongo/Redis URIs, virtual threads enabled, three API keys from env vars with dev defaults, Open Cloud key/universe placeholders, actuator exposure.
CoreApplication.java	Entry point. Three lines: main() calls SpringApplication.run().
common/PingController.java	GET /api/ping returns {"service":"shayveri-core","status":"ok"}.
common/ApiKeyAuthFilter.java	OncePerRequestFilter reading the X-API-Key header and mapping it to a role.
common/ApiKeyProperties.java, ApiKeyRole.java	Loads the three keys from config; ROBLOX / NODE / DASH role enum.
config/SecurityConfig.java	Stateless, CSRF off, /actuator/health public, everything else requires a valid key.
README.md	The full build plan (system context, all 7 module specs, milestones) as project docs.
.gitignore	Excludes .gradle/, build/, .idea/, *.iml and other generated files.

Package layout (package-by-module, not by layer)

```
dev.shayveri.core  
|-- config/      Spring config: security, later WebSocket/Mongo/Redis  
|-- common/     shared DTOs, error handling, API-key auth, /api/ping  
|-- ingress/    Module 1 - telemetry intake from Roblox      (empty)  
|-- nodes/      Module 2 - lab node registry + heartbeats    (empty)
```

```

|-- jobs/           Module 3 - pipeline job queue + lifecycle    (empty)
|-- overrides/      Module 4 - game config overrides          (empty)
|-- egress/         Module 5 - Roblox Open Cloud client        (empty)
|-- realtime/       Module 6 - WebSocket broadcast hub       (empty)
`-- observability/  Module 7 - metrics, audit log                             (empty)

```

2. Decisions and Deviations From the Plan

Spring Boot 3.5.0 instead of the pinned 3.3.4. The plan pinned Boot 3.3.4, but that version has dropped out of Spring Initializr's supported range (it now requires 3.5.0 or newer). Boot 3.5.x satisfies the same 'Boot 3.3+' requirement.

Hand-built scaffold instead of the start.spring.io curl command. The hosted Spring Initializr service was returning HTTP 500 errors server-side, even for minimal requests, so the project was scaffolded manually with identical results (same dependencies, layout, and config).

Gradle installed via Homebrew (9.6.1) only to generate the wrapper; the project itself uses the committed wrapper (8.14.2), so the global install is not a build dependency.

3. Security Model

Role	Who	Env var	Dev default
ROBLOX	Roblox game servers	SHAYVERI_KEY_ROBLOX	dev-roblox-key
NODE	Lab node agents	SHAYVERI_KEY_NODE	dev-node-key
DASH	React dashboard	SHAYVERI_KEY_DASH	dev-dash-key

Keys are sent in the X-API-Key header. No sessions, no CSRF, fully stateless. /actuator/health is the only public endpoint.

4. Why the Code Looks 'Empty'

Spring Boot inverts program flow compared to a plain Main.java project: you write small annotated classes (@RestController, @Configuration) and the framework finds them, wires them together, and calls them when HTTP requests arrive. The server loop, socket handling, and JSON parsing all come from the framework dependencies declared in build.gradle.kts. This is the standard style for every Spring Boot backend and was locked in by the plan's stack decisions. Substantial readable logic arrives with Module 1.

The deep folders (.gradle/, build/, gradle/wrapper contents) are build tooling and caches, not source code - they are gitignored and never edited by hand.

5. Verification Status

Verified: ./gradlew compileJava passes cleanly.

Not yet verified: Docker is not installed on this machine, so docker compose up, ./gradlew bootRun, GET /actuator/health, and the key-gated /api/ping check have not been run against live Mongo/Redis. That is the remaining Phase 0 acceptance work.

6. What to Commit

Commit: src/, gradle/wrapper/ (including the jar), gradlew, gradlew.bat, build.gradle.kts, settings.gradle.kts, docker-compose.yml, README.md, .gitignore.

Do not commit (already gitignored): .gradle/, build/, .idea/, *.iml.

Leftover: the old main/ folder (stub Main.java from before the scaffold) is not part of the project and should be deleted or gitignored before the first commit. Note: git does not track empty directories, so the empty module packages will not appear in the repo until their first class lands.

7. Next Step: Phase 1

Module 1 (Ingress - telemetry intake) plus a minimal Module 6 (Realtime WebSocket hub). Definition of done: a fake Luau script (curl loop) posts telemetry and it is visible on a raw WebSocket client. Full endpoint specs are in README.md.